Clustering in High Dimensions

High-dimensional data drive modern analytics. Marketing teams analyze thousands of pixel intensities or text features; security teams mine flow-level network attributes; editorial teams segment articles in sparse term spaces. Richard Bellman introduced the phrase "the curse of dimensionality" to characterize many problems that arise in higher dimensional analysis. There are two aspects to the "curse" that are worth distinguishing. First, the resources required for processing – computer memory, CPU time – often grow exponentially with the number of dimensions. For example, to find the best linear regression model when there are *p* possible predictors requires checking 2^p models, if all are to be checked. Second, in addition to being quantitatively different with high-dimensional data, the problem is often – unexpectedly – *qualitatively* different as well. For example, distances among data points often become less meaningful in certain senses in high-dimensional data. The preceding sentence is nonintuitive, so clarifying its meaning is part of the agenda of these Notes. We shall focus on cluster analysis, which is the natural unsupervised tool for discovering groups. We shall see that, as dimension *p* grows, distances and densities behave strangely, and naïve clustering often disappoints. This part of the Course Notes explains why, shows what works, and demonstrates how to interpret results for business decisions using real, reproducible numbers that you compute.

We shall proceed in four parts: (i) geometry in high dimensions through a simulation; (ii) PCA-first pipelines for clustering (math + intuition); (iii) three business-style cases with actual outputs: Fashion-MNIST (images), RT-IoT2022 (IoT security flows), and BBC news (text); and (iv) practical guidance on sparsity, stability, common pitfalls, and review questions.

1. Why High Dimensions Are Hard: Distance Concentration

Clustering relies on distances: K-means assigns data points to the nearest centroid; GMMs rely on Mahalanobis distances; hierarchical methods build merges from inter-point distances. But In high dimensions, pairwise distances concentrate—becoming similar across pairs—eroding the very signal clustering needs.

To develop some understanding of this behavior, consider two random draws $x_i, x_j \in \mathbb{R}^p$ with independent identically distributed standard normal coordinates. The squared Euclidean distance between the two points is:

$$\left\|x_i - x_j\right\|^2 = \sum_{k=1}^{p} (x_{ik} - x_{jk})^2 \qquad (12.1)$$

Since $x_{ik} - x_{jk} \sim N(0,2)$ [i.e., each coordinate difference has a Normal distribution with mean = 0 and variance = 2], we have:

$$E\left(\left\|x_i - x_j\right\|^2\right) = 2p \qquad (12.2)$$

For the <u>*squared*</u> distances between data points, the relative variation shrinks as *p* grows:

$$\frac{stdev\left(\left\|x_i - x_j\right\|^2\right)}{E\left(\left\|x_i - x_j\right\|^2\right)} = \frac{\sqrt{2p}}{2p} = \frac{1}{\sqrt{2p}} \to 0 \text{ as } \quad p \to \infty \qquad (12.3)$$

A fortiori, for the <u>*distances*</u> between data points, the relative variation must also shrink as *p* grows. For any random variable *X*, the ratio is $\frac{stdev(X)}{Var(X)}$ is called the **coefficient of variation**. It measures the average percentage (proportion) variation of *X*.

Example 12.1 — A Simulation (small *p* vs big *p*)

The R program "simulation big data demo.R" simulates a sample of *n*=5000 independent standard normal variables with *p*=2 and a different sample of *n*=5000 independent standard normal variables with *p*=200. The samples showed strong distance concentration:

```
p = 2   : mean = 1.7876, sd = 0.9262, relative sd = 0.5181
p = 200 : mean = 19.9765, sd = 0.9852, relative sd = 0.0493
```

In 200 dimensions, almost all distances sit near ~20; the geometry flattens. Clustering that depends on raw Euclidean distance in the original space is therefore fragile.

2. PCA Before Clustering: Intuition and Math

The standard fix is to reduce dimensionality first via Principal Components Analysis (PCA), then cluster in the reduced subspace. PCA rotates the coordinate system to align with directions of maximal variance; a small number of components often captures informative variation while trailing components carry noise or redundancy.

Suppose an *n* row (cases) by *p* column (variables) data matrix *X* that has been centered (column means = 0). The sample covariance, eigen-decomposition, and scores are:

$$\frac{1}{n-1} X^T X \qquad (12.4)$$

$$S = V \Lambda V^T \qquad (12.5)$$

$$Z = XV \qquad (12.6)$$

Keep the first *K* principal components (eigenvectors):

$$Z_K = XV_{1:K} \qquad (12.7)$$

Case Studies

In these case studies, we know the correct labels (clusters) into which the cases should be classified. In real unsupervised learning, we do not know the labels but must assign to clusters anyway. However, in these case studies, we are not going to use our knowledge of the labels until AFTER we have clustered the cases. Then we shall compare our clustering with the truth. Such exercises are useful to gauge how well our clustering methods work. There are many different ways to judge the success of classification procedures. The one used here is ARI = average Rand index. When we do not know the correct labels, success may be judged by the Jaccard index, among other methods. Space does not permit discussion of the many alternative methods. For ARI, the range of possible values is 0 (no good) to 1 (perfect).

3. Case Study A — Fashion-MNIST (images → pixels → PCA → K-means)

Business frame: Retailers need to group products by visual similarity: prototypes for assortment planning, QA anomaly detection, and content organization. Fashion-MNIST provides n = 60,000 images (28×28 grayscale; p = 28*28 = 784), labeled into 10 categories (T-shirt/top, trouser, …, ankle boot). Each pixel is a variable.

The PCA variances for the first 20 principal components are:

```
0.2208 0.1441 0.0546 0.0509 0.0405 0.0302 0.0275 0.0231 0.0169 0.0132
0.0117 0.0096 0.0091 0.0086 0.0074 0.0073 0.0066 0.0063 0.0062 0.0058
```

The cumulative variance (information explained) at 50 PCs = 0.8007.

The ARI score by PC count (single K-means per setting): 20 PCs → 0.3339; 40 PCs → 0.3402; 60 PCs → 0.3461. ARI ≈ 0.34 means clearly better than random yet far from perfect—coarse structure (bags, shoes) appears; confusable categories (T-shirt vs shirt vs pullover) blur.

The key R code:

```
X <- as.matrix(readr::read_csv("images_train.csv.gz", col_names = FALSE,
show_col_types = FALSE))
y <- readr::read_csv("labels_train.csv.gz", col_names = FALSE,
show_col_types = FALSE)[[1]]
Z <- prcomp(scale(X), center = TRUE, scale. = FALSE)$x
for (kpcs in c(20,40,60)) {
  km  <- kmeans(Z[,1:kpcs], centers = 10, nstart = 5, iter.max = 500,
algorithm = "Lloyd")
  ari <- mclust::adjustedRandIndex(km$cluster, y)
  cat(sprintf("%2d PCs  ARI = %.4f
", kpcs, ari))
}
```

4. Case Study B — BBC News (text → TF-IDF → PCA → K-means)

Business frame. Editorial and content teams cluster articles to organize sites, dedupe content, and analyze topics. The BBC corpus (five topics) is a standard research dataset; we used a CSV aggregation (news, type) suitable for immediate R use.

The PCA variances for the first 20 principal components are:

```
0.0054 0.0044 0.0042 0.0039 0.0035 0.0031 0.0030 0.0028 0.0027 0.0026
0.0024 0.0023 0.0023 0.0022 0.0022 0.0021 0.0021 0.0020 0.0020 0.0020
```

The cumulative variance (information explained) at 50 PCs = 0.1087.

For the *K*=5 means clustering, the cluster sizes are (195, 282, 1144, 117, 487) with ARI = 0.4009. The 0.40 ARI is strong for unsupervised TF-IDF clustering: sport words bind sports; business/tech overlap more.

The key R code:

```
library(dplyr); library(tidytext); library(Matrix)
bbc <- readr::read_csv("bbc_dataset.csv", show_col_types = FALSE) %>%
        rename(text = news, label = type) %>%
        mutate(doc_id = row_number())
data("stop_words", package = "tidytext")
tokens <- bbc %>%
  tidytext::unnest_tokens(word, text) %>%
  anti_join(stop_words, by = "word") %>%
  filter(!grepl("^[0-9]+$", word))
term_counts <- dplyr::count(tokens, word, sort = TRUE)
top_terms <- term_counts$word[1:5000]
dtm <- tokens %>%
  filter(word %in% top_terms) %>%
  dplyr::count(doc_id, word, name = "n") %>%
  tidytext::bind_tf_idf(word, doc_id, n)
X <- Matrix::sparseMatrix(i = dtm$doc_id, j = match(dtm$word, top_terms),
                          x = dtm$tf_idf, dims = c(max(dtm$doc_id),
length(top_terms)))
pca <- prcomp(scale(as.matrix(X)), center = TRUE, scale. = FALSE)
Z   <- pca$x[,1:50]; km <- kmeans(Z, centers = 5, nstart = 5, iter.max =
500)
mclust::adjustedRandIndex(km$cluster, as.factor(bbc$label))
```

5. Case Study C — RT-IoT2022 (network security flows → PCA → K-means)

Business frame. Security teams monitor flow statistics to detect IoT [IoT = “internet of things”] attacks (e.g., DDoS, brute-force, scans). RT-IoT2022 provides 123,117 rows and 83 features, plus a multi-class Attack_type label (includes ‘normal’).

The data facts: rows = 123,117; cols = 85 (numeric kept = 81). Label detected: Attack_type.

The PCA variances for the first 20 principal components are:

```
0.2220 0.1226 0.0904 0.0792 0.0558 0.0490 0.0425 0.0298 0.0284 0.0249
0.0238 0.0232 0.0210 0.0168 0.0145 0.0140 0.0132 0.0126 0.0122 0.0118
```

The cumulative variance (information explained) at 50 PCs = 0.9988.

For the clustering (50 PCs; K = 12): ARI vs Attack_type = 0.1632. Clusters loosely align with labeled attack types—expected given mixed traffic and K-means' spherical assumption. Use PCA+clustering for exploratory triage; for detection, supervised models or anomaly scoring are stronger.

6. Feature Selection & Sparsity — When Fewer Features Beat Many

PCA **projects** features; sometimes you want to **select** features for interpretability or to sharpen separation.

Filters. Rank features by variance, mutual information, or univariate tests; keep the top m. Filters are fast and transparent, but ignore interactions.

Embedded sparsity (intuition). Some clustering methods learn **feature weights**, emphasizing dimensions that separate clusters while shrinking others toward zero (**sparse K-means**). A simplified formulation is:

$$\max_{\{C_k\}, w} \sum_{j=1}^{p} w_j \ \mathrm{BCSS}_j \text{s.t.} \sum_{j=1}^{p} |w_j| \leq \tau, \ \ w_j \geq 0, \qquad (12.8)$$

where BCSS_j is between-cluster sum of squares on feature j, and w are nonnegative feature weights under a maximum budget constraint τ. Intuition: concentrate weight on a **sparse** set of discriminative variables; down-weight noise. In practice, PCA-then-K-means is the **first line**; when interpretability of variables matters, add **filtering** or **sparse** variants.

7. Stability in Unsupervised Learning — Jaccard vs ARI

ARI compares a clustering to known labels; it is 1 for perfect agreement and has expected value 0 under random partitions. Jaccard compares two clusterings without labels via pair co-membership overlap (intersection over union). For two clusters,

$$\text{Jaccard} = J(A,B) = \frac{\|A \cap B\|}{\|A \cup B\|} \qquad (12.9)$$

That is, the Jaccard measure is the number of cases in common (intersection) divided by the number of cases in one or the other or both (union).

8. What the Case Study Numbers Teach

• Simulation: relative sd shrank from 0.52 (p=2) to 0.05 (p=200) → raw high-p distances concentrate.
• Fashion-MNIST: 50 PCs ≈ 80% variance; ARI improved very little (0.334→0.346) as PCs increased; coarse prototypes appear, fine categories blur.
• BBC: variance spreads thinly; still ARI ~0.40 with simple TF-IDF + PCA + K-means.

• RT-IoT2022: 50 PCs ≈ 99.9% variance; ARI ~0.16 vs Attack_type; clusters reflect regimes rather than taxonomy.

9. Common Misconceptions

1) 'More features always help.' No—distances concentrate; extra noise hurts.
2) 'Explained variance guarantees good clusters.' No—PCA maximizes variance, not separability.
3) 'K-means failed ⇒ no clusters.' Not necessarily; consider non-spherical structures.
4) 'ARI must be near 1 to be useful.' Not in unsupervised settings; ARI 0.2–0.5 can be managerially useful.

10. Conceptual Review Questions

Q1. Why does Euclidean distance become unreliable in high dimensions? Strong answer: Law-of-large-numbers effect on sums of squared differences; distance concentration (Eqs. 12.1–12.3).

Q2. Why use PCA before clustering? Strong answer: PCA rotates to variance-maximizing axes (Eqs. 12.4–12.7); reduces noise/collinearity; stabilizes geometry.

Q3. Interpret ARI = 0.34 on Fashion-MNIST. Strong answer: Better than random (expected ARI = 0 for random partitions), yet far from perfect; clusters capture coarse visual groups.

Q4. When prefer Jaccard over ARI? Strong answer: When no ground-truth labels exist; Jaccard compares clusterings via co-membership (Eq. 12.9).

Q5. Why can ARI be low in IoT flows though clusters help? Strong answer: Labels reflect fine-grained attack taxonomy; K-means finds broader traffic regimes (bursty vs steady).

References (data & metrics)

• Fashion-MNIST dataset overview & files: https://github.com/zalandoresearch/fashion-mnist
• Fashion-MNIST CSV mirror: https://github.com/insop/Fashion-MNIST-csv
• RT-IoT2022 (UCI Machine Learning Repository): https://archive.ics.uci.edu/dataset/942/rt-iot2022
• RT-IoT2022 single-CSV mirror (teaching convenience): https://huggingface.co/datasets/michaelmallari/rt-iot2022
• BBC datasets (UCD overview): http://mlg.ucd.ie/datasets/bbc.html
• Adjusted Rand Index (mclust documentation): https://mclust-org.github.io/mclust/reference/adjustedRandIndex.html
• Jaccard index (definition): https://en.wikipedia.org/wiki/Jaccard_index